

Amortising an Exact Look-Ahead: Learned Real-Time Policies for Charging-and-Break Scheduling of Electric Trucks

Author Name

Affiliation

name@institution.no

Abstract

Battery-electric trucks must schedule charging jointly with the rest and break periods that driving-hours regulation imposes, under stochastic travel times. The strongest implementable policy we have for this problem is a rolling-horizon look-ahead that solves its tail exactly — and it costs 70 s per decision, which makes it a *batch* policy rather than an operational one. We ask how much of it survives amortisation into a network that decides in 0.5 ms. Behaviour cloning reaches a 3.62% gap to the hindsight optimum against the look-ahead’s 2.22% and a myopic heuristic’s 5.75%; reinforcement fine-tuning closes that to **2.93%**, i.e. **80% of the heuristic-to-solver gap at $\sim 5 \times 10^4$ less online compute**. Getting there required repeatedly trusting closed-loop measurements over offline ones, and the sharpest example is our own: a headline that the learned policy *beat* its teacher, with a reassuring ± 0.05 error bar over training seeds, dissolved under a paired per-instance test into parity. We argue that in sequential operational settings, offline predictive accuracy and seed-level error bars are not merely weak proxies for policy quality — they are routinely *anti-correlated* with it, and reporting them alone is how learned policies come to be overclaimed.

1 Problem and motivation

A battery-electric heavy truck on a long corridor faces two coupled schedules. Charging is dictated by energy and by where stations are; rest and break periods are dictated by EU driving-hours regulation [4], on a corridor whose station spacing is itself set by regulation [5]. The two interact: a mandatory break taken *inside* a charging dwell is free, because the two run in parallel, so *when* to charge is inseparable from *when* to rest. Travel times are stochastic, so the schedule must be decided online, stop by stop.

Our companion study benchmarks a family of policies on this problem — a myopic heuristic, static robust plans, a two-stage stochastic plan with recourse, a rolling-horizon look-ahead, and a per-realisation hindsight oracle. Here we take that benchmark as given and ask a narrower question about its best implementable member.

Where the remaining headroom is. A one-at-a-time sweep of the look-ahead’s parameters is uninformative in two of its three dimensions. Raising the scenario count from 10 to 50 moves realised duration by at most 0.4%; extending the horizon beyond one duty cycle (24 h) by at most 0.2%, although *shortening* it below one duty cycle costs up to 1.7%. The one live axis is **integrality**: solving the look-ahead tail as a MIP rather than an LP relaxation roughly halves the gap to the hindsight optimum (e.g. 4.3% $\rightarrow$ 2.1% on short routes without windows, 6.8% $\rightarrow$ 2.2% on medium routes with tight ones) and costs an order of magnitude more time per decision.

The gap this paper attacks. Exactness is therefore worth about two points of optimality and is now the benchmark’s standard configuration — but at 45–90 s per decision and 0.6–4.1 h of wall clock per route it is affordable only as an offline campaign. It cannot be re-evaluated continuously in a cab, run across a fleet by a dispatcher, or deployed without a commercial solver licence. We therefore ask: *how much of an exact look-ahead survives compression into a policy that decides in milliseconds?* In Powell’s taxonomy [8] this converts a direct look-ahead into a policy-function approximation; in the ML-for-combinatorial-optimisation framing [2] it is amortised optimisation [3, 7]. Unlike neural construction heuristics for routing [6] the stop sequence is fixed and the learned object is the operational schedule; the closest application is safe reinforcement learning for dynamic electric vehicle routing [1].

Table 1: Held-out test set (108 instances; common completed set $n = 107$). Oracle gap is on the penalised objective. The learned policies reach solver-comparable quality without a solver and $\sim 5 \times 10^4$ faster.

Policy	Gap to oracle	vs. teacher (h)	Halts /108	TW misses	t_{dec}
Greedy heuristic	+5.75%	—	0	166	0.03 ms
Student (cloned)	+3.62%	+0.283	1.7	135	0.50 ms
Student + PPO	+2.93%	+0.025	4.0	118	0.51 ms
Exact-tail look-ahead	+2.22%	— (reference)	0	53	70,446 ms

2 Method

Decision process. At stop i the state comprises state of charge, the regulatory clocks (consecutive driving, shift driving, shift work, and the 15 h daily *spread*), the split-break flag, budgets consumed, and arrival time; plus the road ahead — the next K nodes (leg duration and energy, node type, queue delay, time-window slack *relative to now*) and aggregates over the whole remaining route. With $K = 20$ this is 143 features. The action is the joint discrete choice over charging, break type and rest type, of which 12 combinations occur in the data, together with a charge duration τ_c . The reward is the negative increment of the benchmark’s own objective (duration plus β per missed window), so training and evaluation optimise the same quantity.

Policy and constraint layers. A two-layer MLP ($143 \rightarrow 128 \rightarrow 128$; 36,621 parameters) with masked logits over the 12 actions and a softplus charge duration. Two constraint layers wrap it, both reusing the benchmark’s own feasibility checks so the learned policy obeys exactly the rules every other policy obeys. *Masking* sets structurally impossible actions to $-\infty$ before the softmax. *Forcing* restricts the choice set when a one-step check declares a charge, break or rest mandatory — the network still chooses freely *among* compliant actions, so the rules constrain rather than decide. Charge duration is clamped below by the energy needed to reach the next station at the 0.5-quantile of consumption, the same rule the teacher uses. **No solver runs at deployment.**

Cloning, then fine-tuning. 767 exact-tail look-ahead runs over a $3 \times 3 \times 4$ grid of route length, customer density and window class supply **74,380** labelled decisions; instances are split by seed into 530 train / 129 validation / 108 test. Behaviour cloning gives the initial policy; a round of DAgger [9] on states the clone itself visits improved validation but did not transfer to the held-out set, so it is not used here. Because imitation is bounded by its teacher by construction, we fine-tune with PPO on the realised objective, initialised from the clone, with a value head on the shared trunk and a KL penalty back to the frozen clone. Training episodes draw *fresh* travel-time realisations — each instance ships one recorded realisation, which evaluation uses, and training on it repeatedly would learn a plan rather than a policy. Only the discrete head is fine-tuned. A halt (any regulatory breach ends the route) is charged twice an estimate of what completing would have cost; Section 3 shows that this shaping choice, not the RL algorithm, is what governs reliability.

3 Experiments

Every policy runs in the same simulator, over the same recorded realisations, scored by the same metrics. A run that breaches a regulatory limit *halts* at that stop and has no duration, so halts are counted, never averaged. We report the gap to the per-realisation hindsight oracle — an absolute yardstick — alongside paired per-instance differences from the teacher. Three independently trained seeds are averaged *per instance* first, so the unit of analysis is the instance rather than the (instance, seed) pair.

Main result. Table 1 and Figure 1 give the headline. Cloning the look-ahead reaches a 3.62% oracle gap; PPO fine-tuning closes this to **2.93%** against the look-ahead’s 2.22%, i.e. **80% of the distance between the myopic heuristic and the exact solver**, while deciding in 0.51 ms and requiring no solver at all.

But it does not beat its teacher, and the error bar said it did. Across three seeds the fine-tuned policy’s median duration difference was $-0.24 \pm 0.05\%$ — negative on every seed, with a tight spread. Read literally, the student beat a 70-second solver. It does not. That $\pm$ measures *seed reproducibility*, not whether the policy is better: seed spread (0.05)

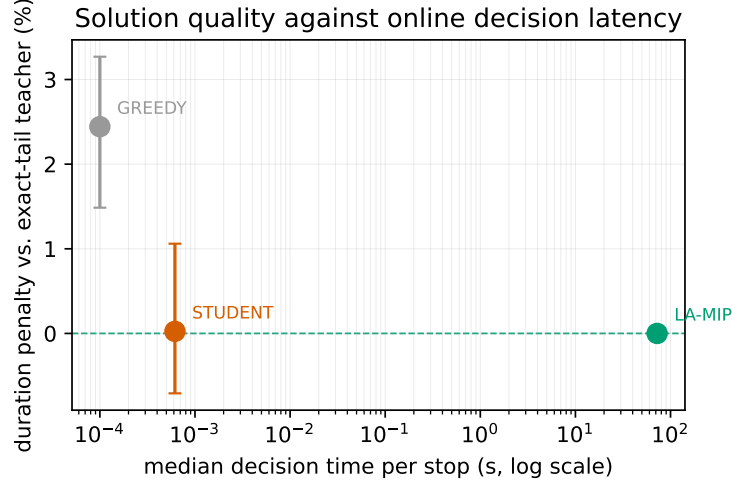


Figure 1: Quality against online decision latency (log scale). The learned policies attain solver-comparable quality at heuristic latency. Bars are interquartile ranges over instances.

Table 2: Design decisions (validation). In both rows the offline metric prefers the worse policy.

Decision	Setting	Balanced acc. $\uparrow$	Duration vs. teacher $\downarrow$
Class weighting	inverse-frequency ($p=0.5$)	0.477	+3.17%
	unweighted ($p=0$)	0.299	+0.56%
Look-ahead features	$K = 40$ (261 features)	0.452	+2.86%
	$K = 20$ (143 features)	0.299	+0.56%

is an order of magnitude smaller than instance-to-instance spread. A paired per-instance test on validation puts the 95% bootstrap CI of the median at $[-0.17, +0.05]$ h with the policy ahead on 54% of instances (exact sign test $p = 0.19$); on the test set it is ahead on 45% ($p = 0.88$). **The honest claim is parity, not superiority**, and detecting a 54% win rate would need on the order of a thousand instances against the 237 uncontaminated ones we have. We report this at length because the mistake was ours, it survived three seeds, and nothing in the offline metrics would have caught it.

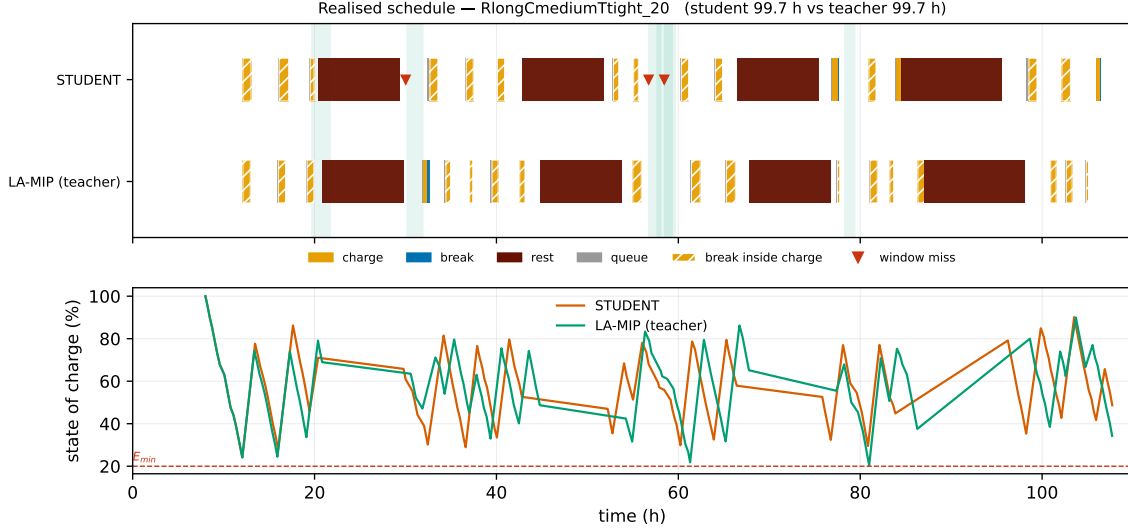
Ablations: the offline metric prefers the worse policy. Table 2 pairs each design decision’s supervised metric with its realised effect. Inverse-frequency class weighting is the clearest case. The decision distribution is dominated by “keep driving” (87.9%), so weighting rare classes up is the textbook response, and by its own measure it works — balanced accuracy rises from 0.299 to 0.477. But it places the majority class at weight 0.035 against 5.3 for the rarest, so predicting a *rest* when the truth is *drive on* becomes nearly free in the loss while costing 9–11 h of route time in reality; realised duration degrades sixfold. The loss had decorrelated from the objective, and no offline metric revealed it.

What made reinforcement learning work. A first PPO run improved duration but raised halts by 70%, and 27 of its 29 failures breached the 15 h daily-spread ceiling — a limit the policy *could not observe*, because the spread clock was absent from the state. Two changes followed: adding it (and its headroom) as features, and replacing a fixed halt surcharge with one proportional to the estimated cost of completing. Rerunning with one change at a time separates their roles cleanly (Table 3): the feature supplies the quality, the penalty supplies the reliability, and with the original additive penalty reinforcement learning does not reduce halts at all.

Behavioural fidelity, and what is still missing. Figure 2 shows a realised schedule against the teacher’s: rest placement, charging episodes and the state-of-charge trajectory are essentially superimposed, and both policies hide almost every mandatory break inside a charging dwell — the coupling that makes this problem non-separable. The learned policy charges 0.9% *less* than the teacher and runs the pack to the same depth, so it has learned that charge is time

Table 3: Which reinforcement-learning fix did what (validation, 3 seeds).

Variant	Halts /129	Gap to oracle	vs. teacher (h)
Cloned policy (no RL)	4.0	+4.00%	+0.274
PPO, additive halt penalty	4.3	+3.11%	−0.036
PPO, multiplicative penalty	1.7	+3.00%	−0.088

**Figure 2:** Realised schedule, student (top lane) against exact-tail teacher (bottom lane), on a held-out long route with tight windows. Hatched charging bars are dwells that absorb a mandatory break at no extra cost. Lower panel: state of charge.

rather than merely that empty batteries are bad. Time windows remain the deficiency (118 misses against 53), which is why the penalised gap closes less than the duration gap; halts also do not transfer consistently, being lower than the clone’s on validation and higher on test.

4 Discussion

An exact-tail look-ahead can be compressed into a 36k-parameter network that decides $\sim 5 \times 10^4$ times faster and gives up about seven tenths of a point of oracle gap — the difference between a policy that can be run as an offline study and one that can be run in a vehicle.

The transferable lesson is methodological, and it has two halves. The first is familiar: offline predictive accuracy repeatedly preferred the worse policy, because in sequential operational problems the cost of an error is wildly non-uniform across the action space — a misplaced daily rest costs nine hours, a misplaced charging decision costs minutes — and any loss that treats them comparably has stopped measuring the objective. The second is less often stated. Having moved to closed-loop cost, we still overclaimed, because we reported variability across training seeds. That quantity is easy to compute, looks like an error bar, and answers the wrong question; the instance-level uncertainty that decides whether one policy beats another was an order of magnitude larger. We would encourage reporting realised closed-loop cost against an absolute yardstick, with paired per-instance tests, and treating seed spread as a reproducibility diagnostic rather than a confidence interval.

Limitations. Single vehicle; synthetic travel-time uncertainty from a fitted shifted lognormal; three training seeds; the test set was evaluated twice, once for an earlier feature set and once for the final system, with all tuning confined to validation. Time windows are the standing deficiency and reinforcement learning did not repair them despite their being priced in the reward, which suggests the signal is too weak relative to duration. Extending to a fleet sharing charging infrastructure is the natural next step, and turns a single-agent problem into a multi-agent one.

References

- [1] Rafael Basso, Balázs Kulcsár, Ivan Sanchez-Diaz, and Xiaobo Qu. Dynamic stochastic electric vehicle routing with safe reinforcement learning. *Transportation Research Part E*, 157, 2022.
- [2] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: A methodological tour d’horizon. *European Journal of Operational Research*, 290(2):405–421, 2021.
- [3] Justin Dumouchelle, Rahul Patel, Elias B. Khalil, and Merve Bodur. Neur2SP: Neural two-stage stochastic programming. In *NeurIPS*, 2022.
- [4] European Parliament and Council. Regulation (ec) no 561/2006 on the harmonisation of certain social legislation relating to road transport. 2006.
- [5] European Parliament and Council. Regulation (eu) 2023/1804 on the deployment of alternative fuels infrastructure. 2023.
- [6] Wouter Kool, Herke van Hoof, and Max Welling. Attention, learn to solve routing problems! In *ICLR*, 2019.
- [7] Vinod Nair, Sergey Bartunov, Felix Gimeno, et al. Solving mixed integer programs using neural networks. *arXiv:2012.13349*, 2020.
- [8] Warren B. Powell. *Reinforcement Learning and Stochastic Optimization: A Unified Framework for Sequential Decisions*. Wiley, 2022.
- [9] Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *AISTATS*, 2011.